

Computer Networks II

TCP Congestion and Flow Control

Other Variants, TCP Fairness, Flow Control

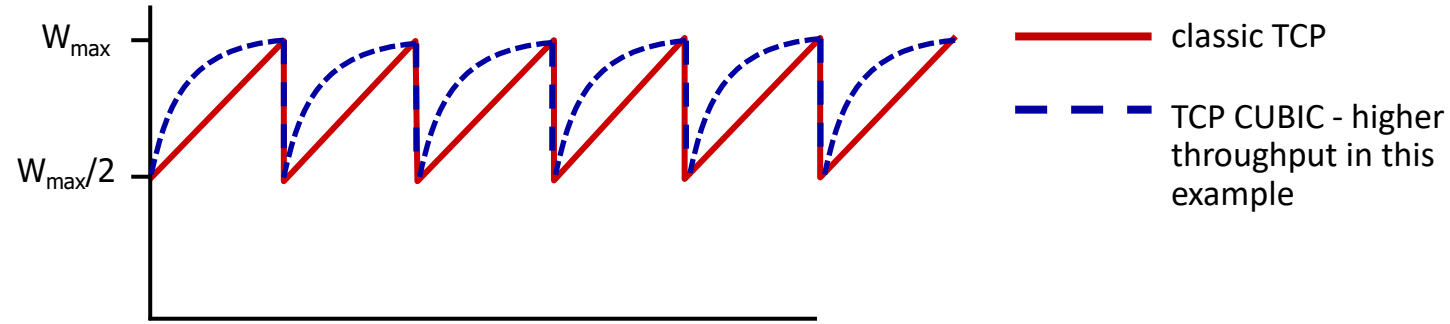
Amitangshu Pal

Computer Science and Engineering

IIT Kanpur

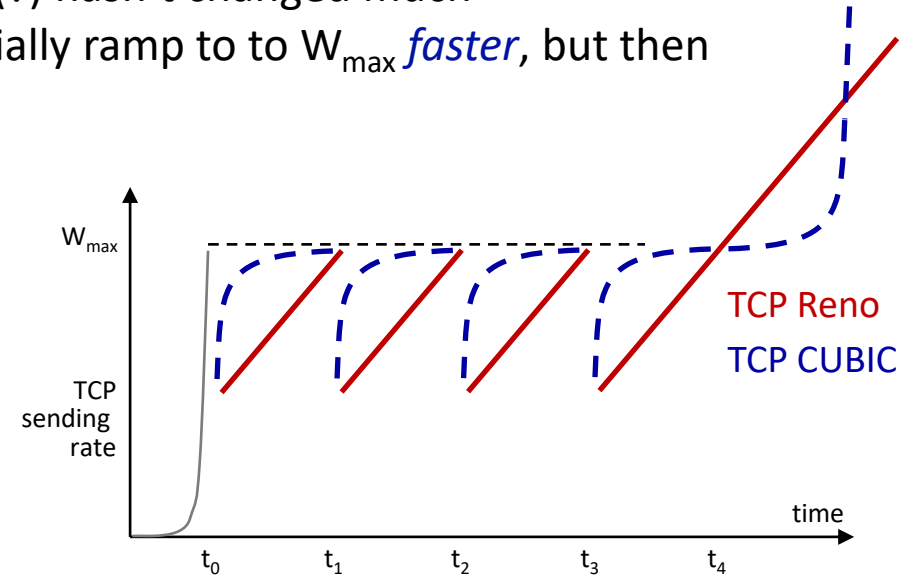
TCP CUBIC

- Is there a better way than AIMD to “probe” for usable bandwidth?
- Insight/intuition:
 - $W_{\max}$: sending rate at which congestion loss was detected
 - congestion state of bottleneck link probably (?) hasn't changed much
 - after cutting rate/window in half on loss, initially ramp to to $W_{\max}$ *faster*, but then approach $W_{\max}$ more *slowly*



TCP CUBIC

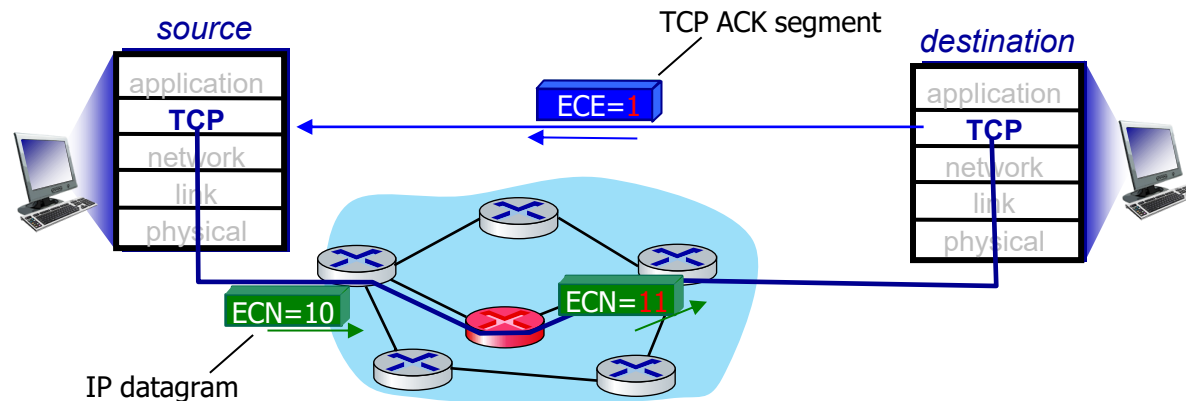
- Is there a better way than AIMD to “probe” for usable bandwidth?
- Insight/intuition:
 - $W_{\max}$: sending rate at which congestion loss was detected
 - congestion state of bottleneck link probably (?) hasn't changed much
 - after cutting rate/window in half on loss, initially ramp to to $W_{\max}$ *faster*, but then approach $W_{\max}$ more *slowly*
- TCP CUBIC default in Linux, most popular TCP for popular Web servers



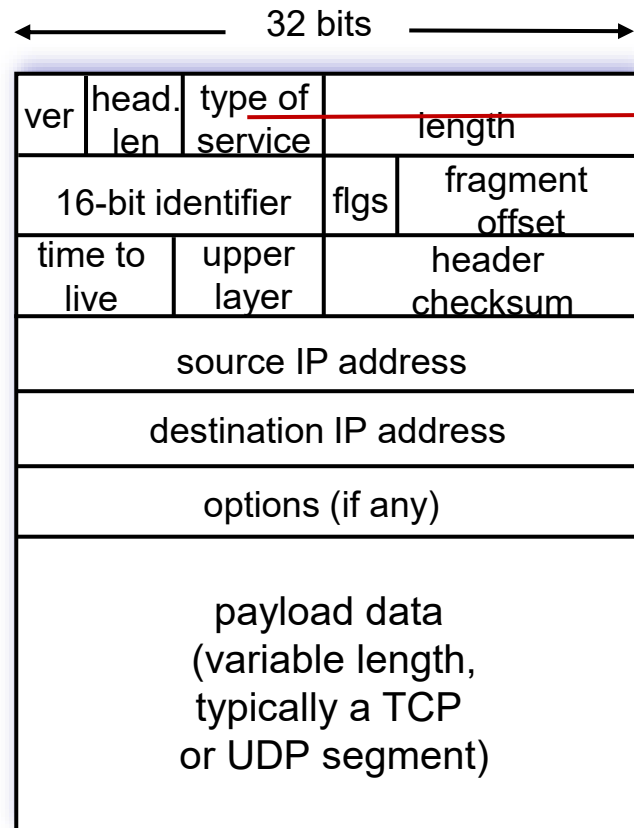
Explicit congestion notification (ECN)

TCP deployments often implement *network-assisted* congestion control:

- two bits in IP header (ToS field) marked *by network router* to indicate congestion
 - *policy* to determine marking chosen by network operator
- congestion indication carried to destination
- destination sets ECE bit on ACK segment to notify sender of congestion
- involves both IP (IP header ECN bit marking) and TCP (TCP header C,E bit marking)



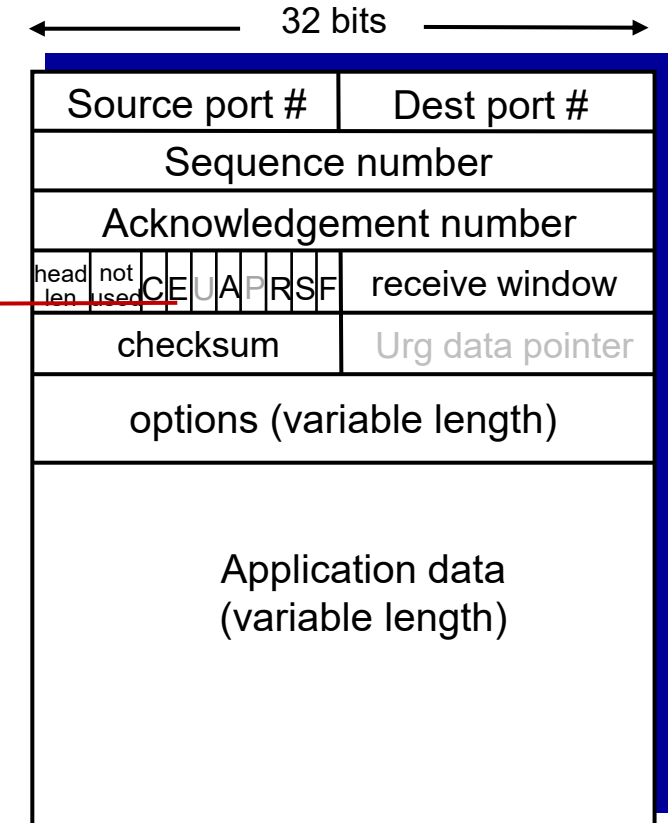
Explicit congestion notification (ECN)



“type” of service:

- diffserv (0:5)
- ECN (6:7)

C, E:
congestion notification

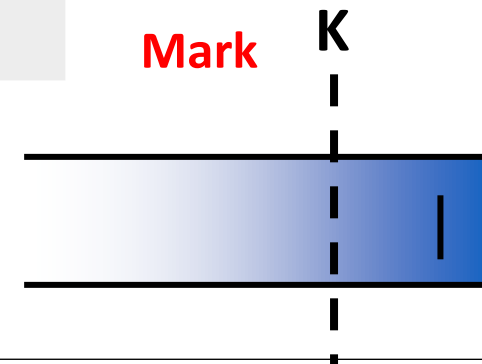


DCTCP: Two Key Ideas

1. React in proportion to the **extent** of congestion, not its **presence**.
 - ✓ Reduces **variance** in sending rates, lowering queuing requirements.

ECN Marks	TCP	DCTCP
1 0 1 1 1 1 0 1 1 1	Cut window by 50%	Cut window by 40%
0 0 0 0 0 0 0 0 0 1	Cut window by 50%	Cut window by 5%

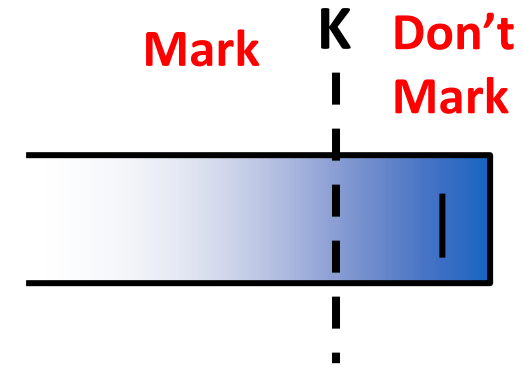
2. Mark based on **instantaneous** queue length.
 - ✓ Fast feedback to better deal with bursts.



Data Center TCP Algorithm

Switch side:

- Mark packets when **Queue Length > K**



Sender side:

- Maintain running average of **fraction** of packets marked (α).

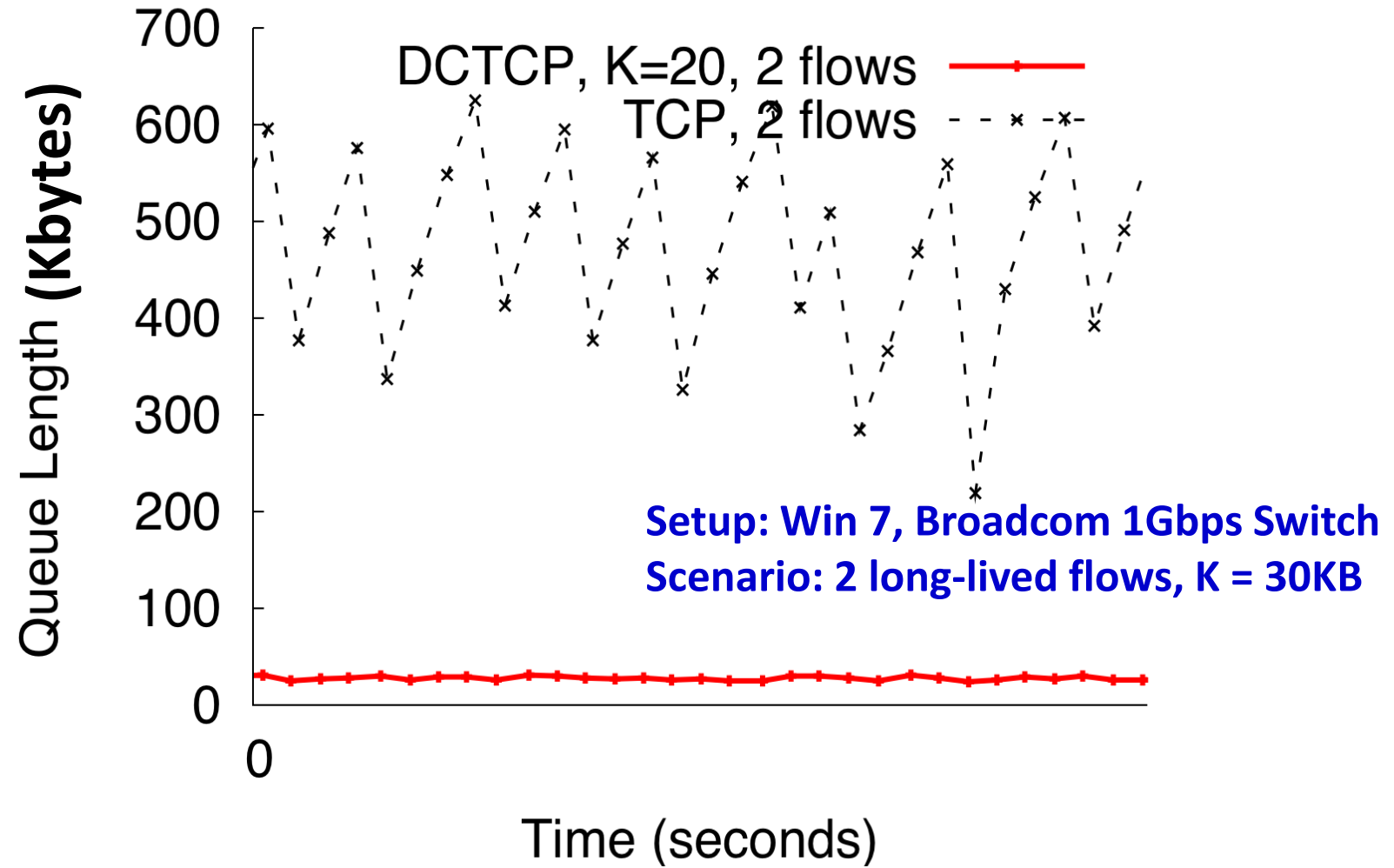
In each RTT:

$$F = \frac{\# \text{ of marked ACKs}}{\text{Total \# of ACKs}}$$

$$\alpha \leftarrow (1 - g)\alpha + gF$$

➤ Adaptive window decreases: $cwnd \leftarrow (1 - \frac{\alpha}{2})cwnd$

DCTCP in Action



RTO Calculation

TCP round trip time, timeout

Q: how to set TCP timeout value?

- longer than RTT, but RTT varies!
- *too short*: premature timeout, unnecessary retransmissions
- *too long*: slow reaction to segment loss

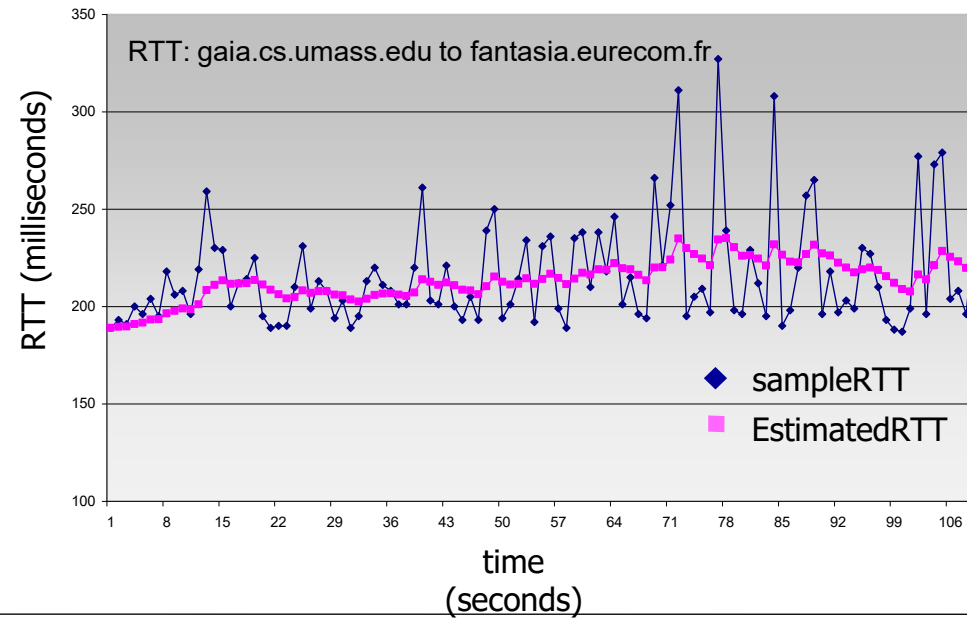
Q: how to estimate RTT?

- `SampleRTT`: measured time from segment transmission until ACK receipt
 - ignore retransmissions
 - `SampleRTT` will vary, want estimated RTT “smoother”
 - average several *recent* measurements, not just current `SampleRTT`
-

TCP round trip time, timeout

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- exponential weighted moving average (EWMA)
- influence of past sample decreases exponentially fast
- typical value: $\alpha = 0.125$



TCP round trip time, timeout

- **DevRTT**: EWMA of **SampleRTT** deviation from **EstimatedRTT**:

$$\text{DevRTT} = (1-\beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

(typically, $\beta = 0.25$)

- timeout interval: **EstimatedRTT** plus “safety margin”
 - large variation in **EstimatedRTT**: want a larger safety margin

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$



↑
estimated RTT

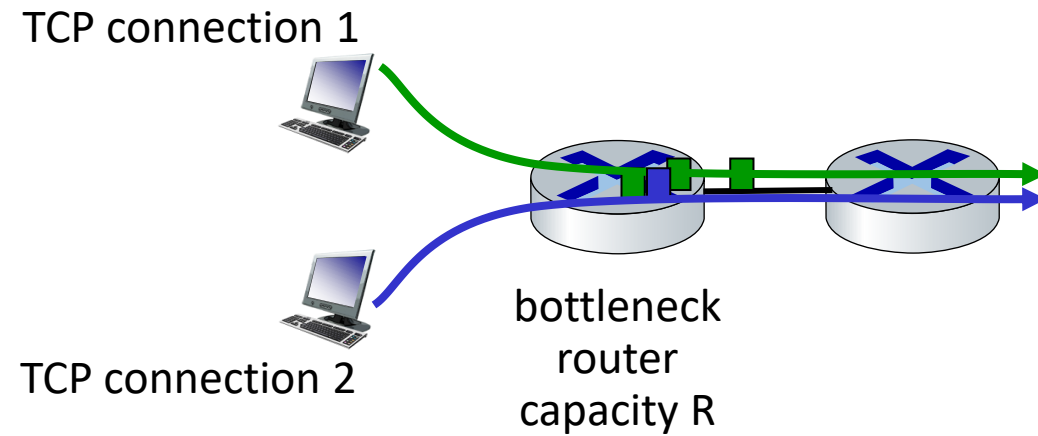
↑
“safety margin”

More reading: <http://research.protocollabs.com/captcp/doc-socket-statistic-module.html>

TCP Fairness

TCP fairness

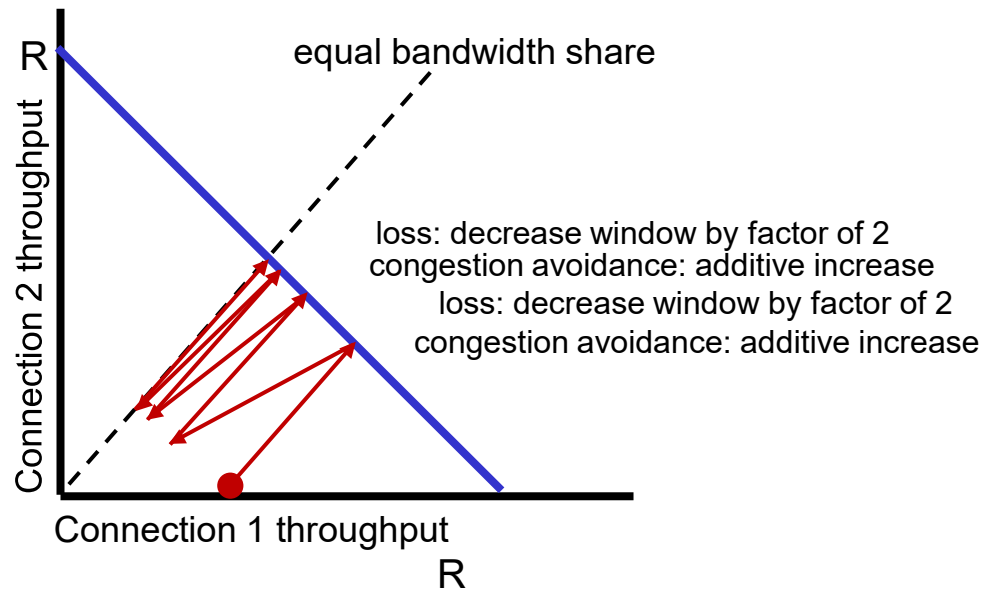
Fairness goal: if K TCP sessions share same bottleneck link of bandwidth R , each should have average rate of R/K



Is TCP Fair?

Example: two competing TCP sessions:

- additive increase gives slope of 1, as throughput increases
- multiplicative decrease decreases throughput proportionally



Is TCP fair?

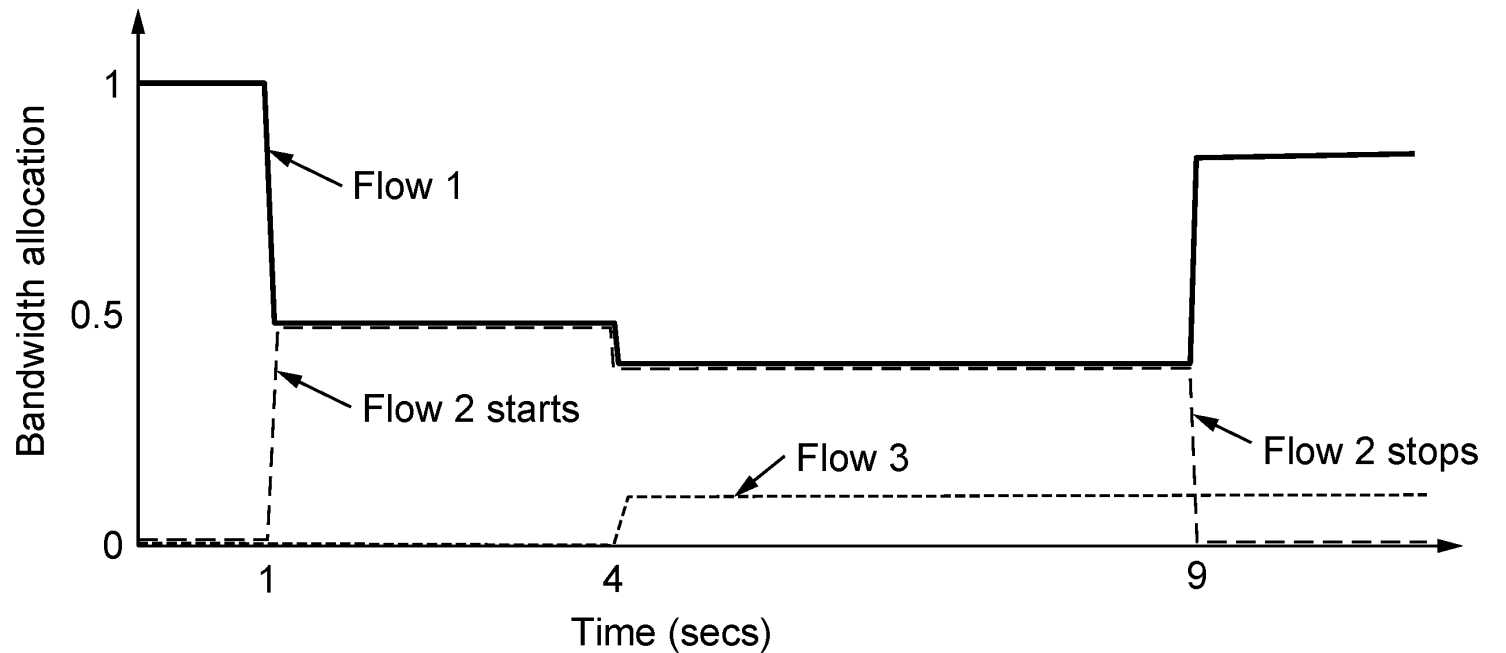
A: Yes, under idealized assumptions:

- same RTT
- fixed number of sessions only in congestion avoidance

Is TCP Fair?

Example: two competing TCP sessions:

- additive increase gives slope of 1, as throughput increases
- multiplicative decrease decreases throughput proportionally



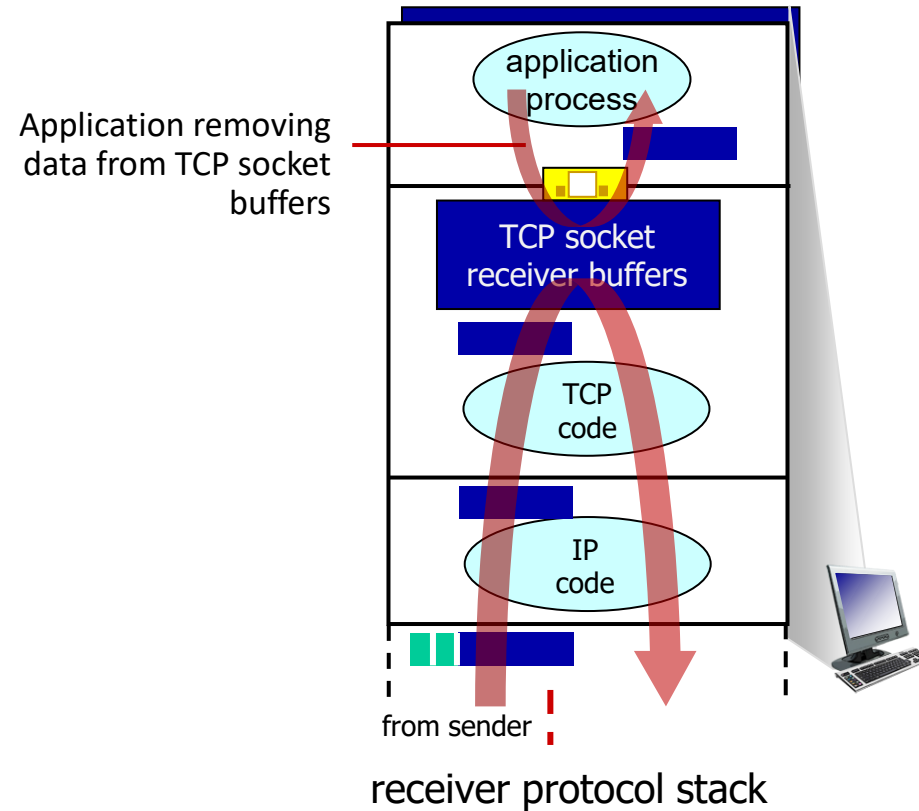
TCP Flow Control

TCP flow control

Q: What happens if network layer delivers data faster than application layer removes data from socket buffers?

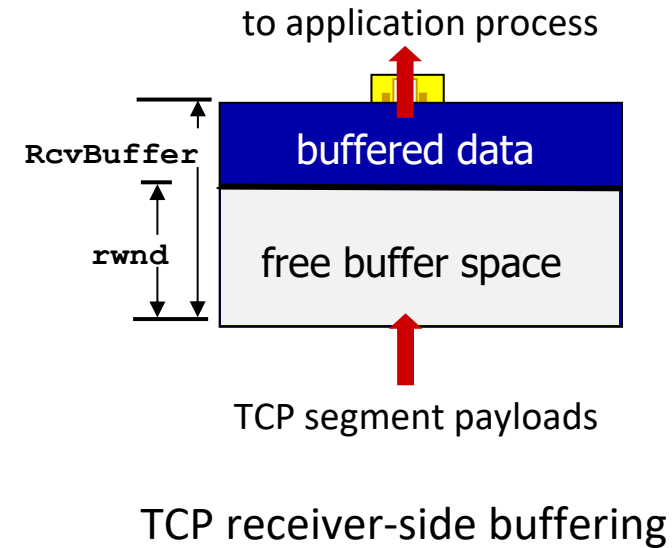
flow control

receiver controls sender, so sender won't overflow receiver's buffer by transmitting too much, too fast



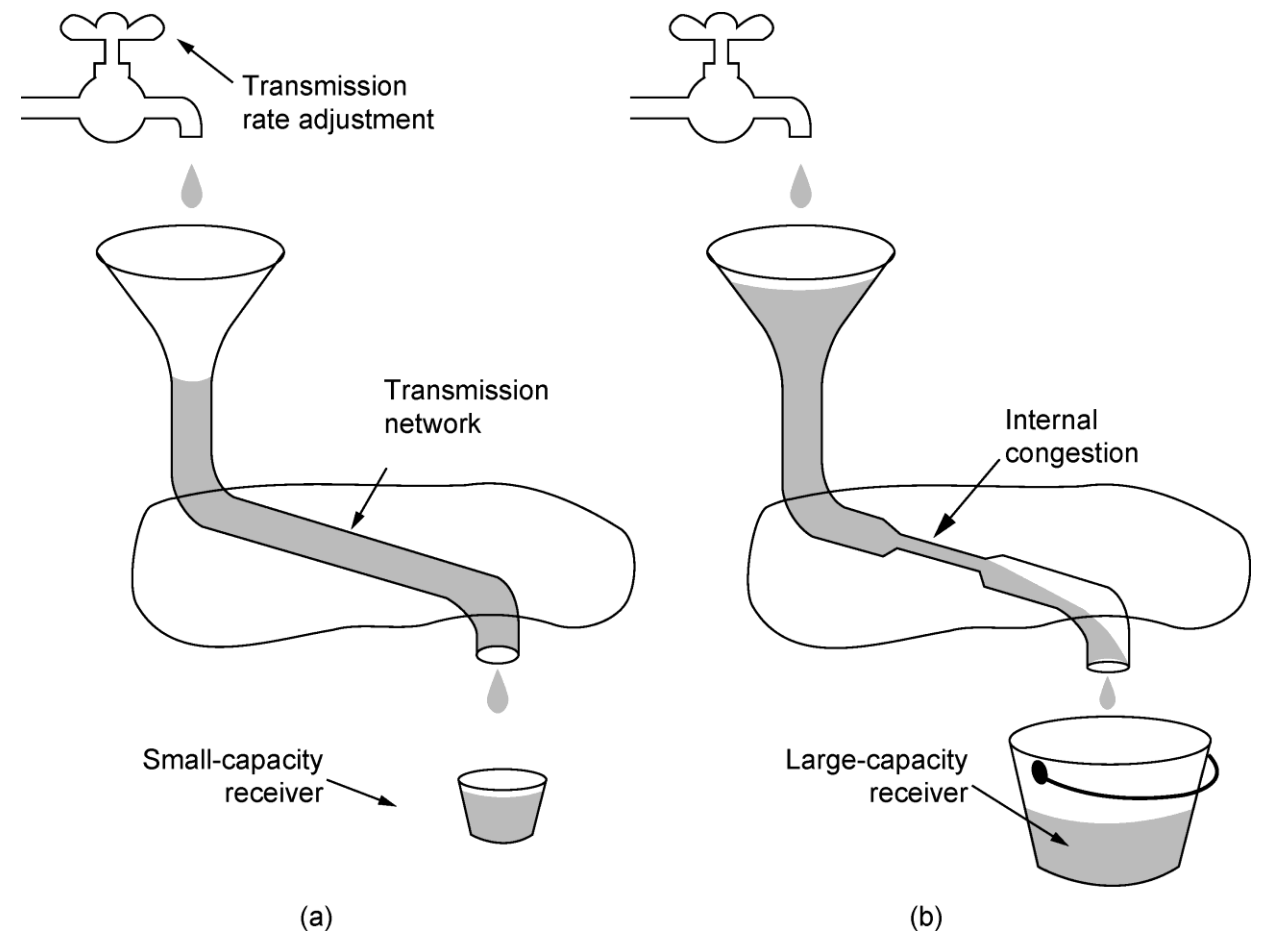
TCP flow control

- TCP receiver “advertises” free buffer space in **rwnd** field in TCP header
 - **RcvBuffer** size set via socket options (typical default is 4096 bytes)
 - many operating systems autoadjust **RcvBuffer**
- sender limits amount of unACKed (“in-flight”) data to received **rwnd**
- guarantees receive buffer will not overflow



TCP flow control

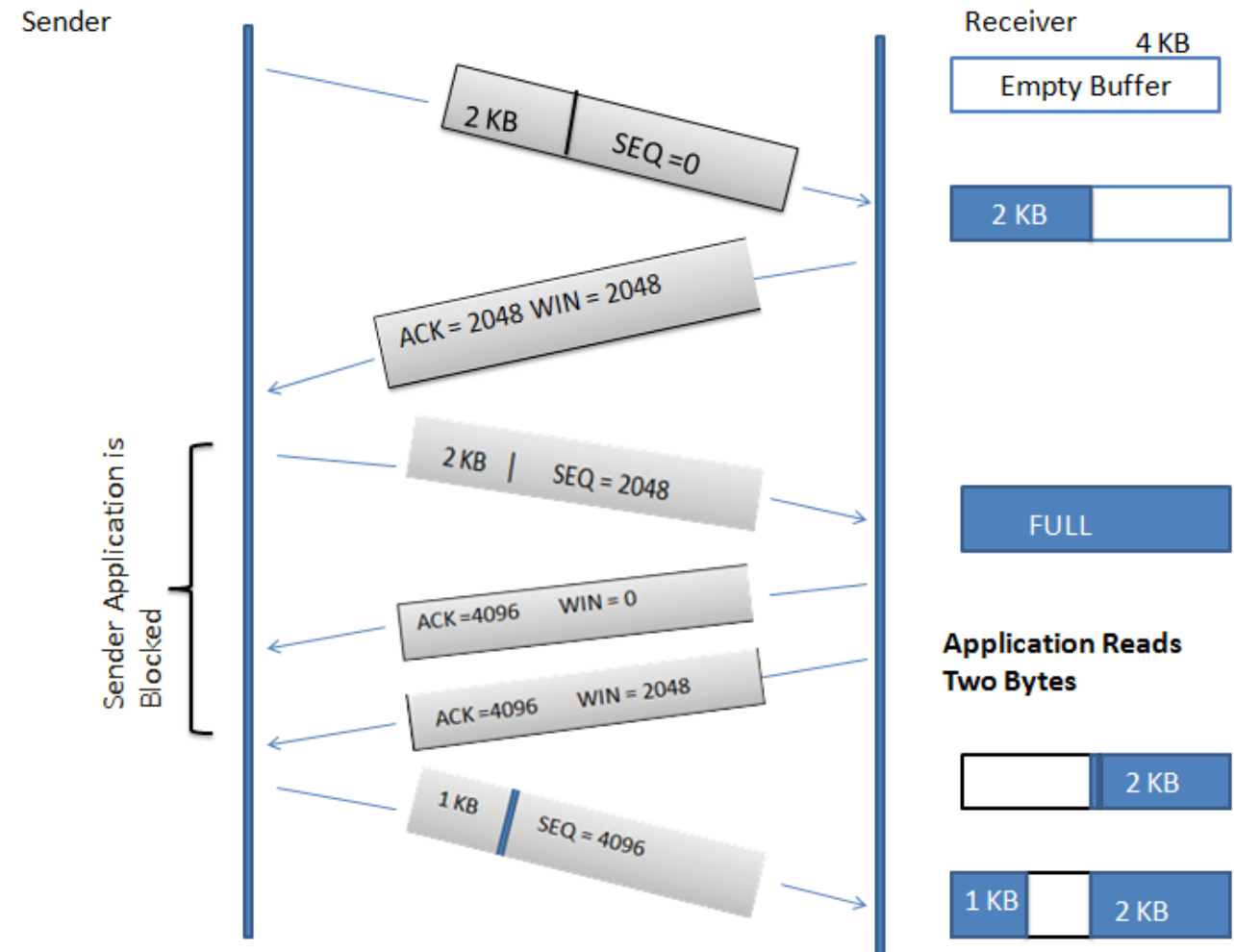
- TCP receiver “advertises” free buffer space in **rwnd** field in TCP header
 - **RcvBuffer** size set via socket options (typical default is 4096 bytes)
 - many operating systems autoadjust **RcvBuffer**
- sender limits amount of unACKed (“in-flight”) data to received **rwnd**
- guarantees receive buffer will not overflow



- (a) A fast network feeding a low-capacity receiver.
(b) A slow network feeding a high-capacity receiver.

TCP flow control

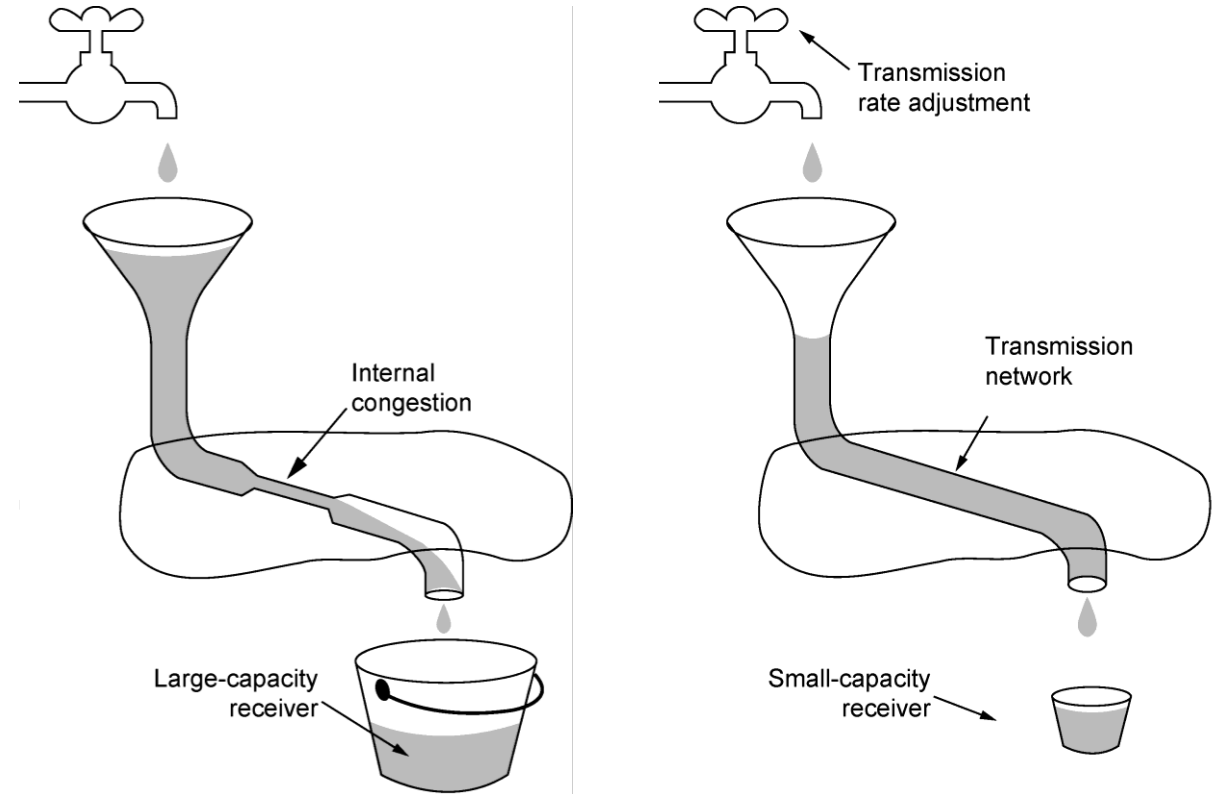
- TCP receiver “advertises” free buffer space in **rwnd** field in TCP header
 - **RcvBuffer** size set via socket options (typical default is 4096 bytes)
 - many operating systems autoadjust **RcvBuffer**
- sender limits amount of unACKed (“in-flight”) data to received **rwnd**
- guarantees receive buffer will not overflow



Src: https://upload.wikimedia.org/wikipedia/commons/5/57/Flow_Control_in_TCP.png

TCP Congestion Control vs Flow Control

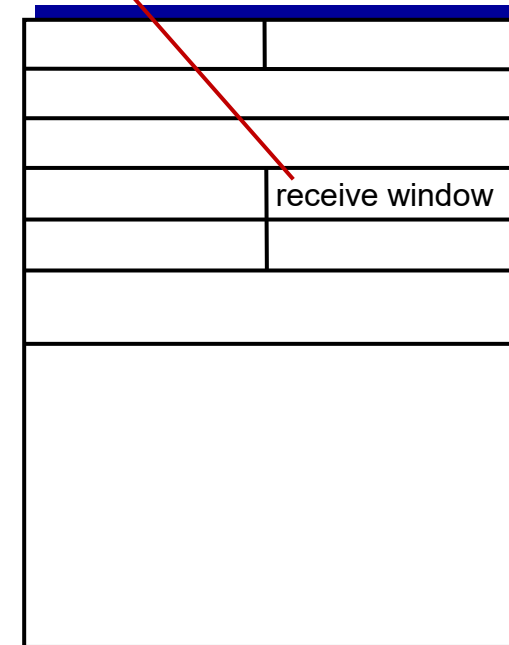
- **Congestion control**
 - Sender will not overwhelm the network
- **Flow control**
 - Sender will not overwhelm the receiver



TCP flow control

- TCP receiver “advertises” free buffer space in **rwnd** field in TCP header
 - **RcvBuffer** size set via socket options (typical default is 4096 bytes)
 - many operating systems autoadjust **RcvBuffer**
- sender limits amount of unACKed (“in-flight”) data to received **rwnd**
- guarantees receive buffer will not overflow

flow control: # bytes receiver willing to accept



TCP segment format